

IoT Challenge #3 – Part 1 (Challenge)

Tommaso Neri

Person Code: 10817232

Individual submission

May 6, 2026

Overview

The implemented Node-RED flow processes the dataset `challenge3.csv` using the MQTT-based identifier generator required by the assignment. The flow uses a local Mosquitto broker on `localhost:1884`, publishes and subscribes to the topic `challenge3/id_generator`, extracts the CSV row identified by:

$$N = ID \bmod 5218,$$

and handles the two required packet classes:

- packets whose **Command String** contains a **ZBEE_ZCL** layer;
- packets whose **Packet Type** is **Link Status** (0x08).

All other packets are ignored, but they still count toward the limit of 200 received ID messages. The processing stop condition is implemented in the subscriber branch. The generator is also stopped automatically after the subscriber completes the 200 received IDs, in order to avoid producing an oversized `id_log.csv`.

Node-RED Flow

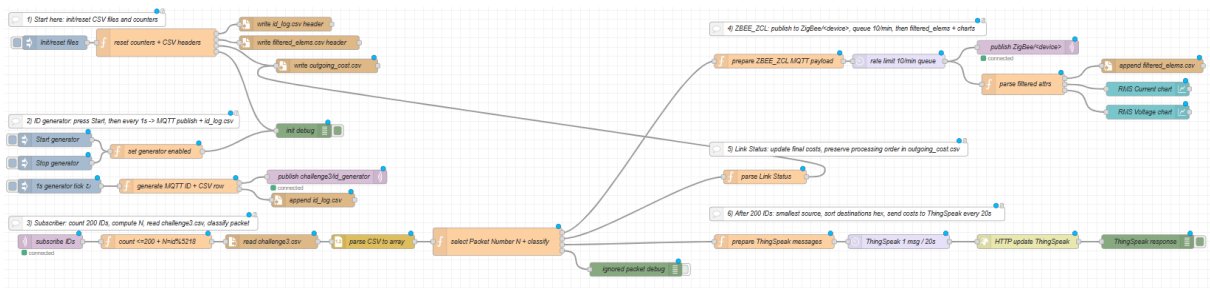


Figure 1: Complete Node-RED flow used for Challenge 3.

The flow is organized into six main blocks, matching the structure of the assignment: initialization, ID generation, ID subscription and CSV selection, ZBEE_ZCL publishing and filtered attributes, Link Status processing, and ThingSpeak updates.

Initialization

The `Init/reset` files inject node triggers the function `reset counters + CSV headers`. This function resets all flow variables used during the execution:

- `id_log_no`, used as row counter for `id_log.csv`;
- `id_rx_count`, used to enforce the 200 received ID limit;
- `filtered_no`, used as row counter for `filtered_elems.csv`;
- `link_costs` and `link_order`, used to store final Link Status costs while preserving processing order;
- `challenge_done`, used to stop processing after 200 received IDs;
- `thingspeak_started`, used to start the ThingSpeak branch only once;
- `generator_enabled`, used to manually start or stop the ID generator.

The same function also rewrites the headers of the three output CSV files: `id_log.csv`, `filtered_elems.csv`, and `outgoing_cost.csv`.

ID Generator Branch

The ID generator is controlled by the `Start generator` and `Stop generator` inject nodes. A periodic inject node named `1s generator tick` fires once per second. The function `generate MQTT ID + CSV row` checks whether generation is enabled and, if so, creates:

- a random ID in the range `[0, 30000]`;
- the current UNIX timestamp;
- the MQTT payload `{"id": ID, "timestamp": TIMESTAMP}`;
- one row for `id_log.csv`.

The MQTT message is published through the `mqtt out` node on:

`challenge3/id_generator.`

The CSV row is appended to `id_log.csv` with the required format:

`No., ID, TIMESTAMP.`

Subscriber and CSV Selection Branch

The `mqtt` in node subscribes to `challenge3/id_generator` on the local broker. The function `count <=200 + N=id%5218` parses the JSON payload, extracts the ID, increments the subscriber-side counter, and computes:

$$N = ID \bmod 5218.$$

When the received ID counter reaches 200, the flow variable `challenge_done` is set. Further messages are not processed.

The `file` in node reads `challenge3.csv`, and the `csv` node parses it as an array of rows. The function `select Packet Number N + classify` selects the row whose `Packet Number` is equal to N and routes the message as follows:

- to the `ZBEE_ZCL` branch if `Command String` contains `ZBEE_ZCL`;
- to the `Link Status` branch if `Packet Type` is `Link Status (0x08)`;
- to a disabled debug node otherwise.

ZBEE_ZCL Publish Branch

For `ZBEE_ZCL` packets, the function `prepare ZBEE_ZCL MQTT payload` builds the MQTT topic:

`ZigBee/<Device Name ZigBee Source>`.

The same topic is assigned both to `msg.topic` and to the `topic` field inside the JSON payload, as required by the assignment clarification.

The published payload contains:

- current UNIX timestamp;
- ID received by the subscription;
- WPAN source and destination addresses;
- ZigBee source and destination addresses;
- MQTT topic;
- original `Command String`.

The `delay` node named `rate limit 10/min queue` limits this branch to 10 messages per minute. It is configured in queue mode, so messages are delayed but not dropped. The rate-limited messages are sent to the local MQTT broker through the `publish ZigBee/<device>` node.

Filtered Attributes and Dashboard Charts

After the ZBEE_ZCL rate limiter, the function `parse filtered attrs` parses the command string and searches for the required attributes:

- Active Power;
- RMS Current;
- RMS Voltage.

The function keeps the positional association required by the assignment:

`Attribute[i] ↔ Status[i] ↔ Data Type[i] ↔ Data Value[i]`.

Only complete attributes are saved. If the corresponding status, data type, or value is missing, that attribute is excluded and no empty CSV cell is written.

Valid rows are appended to `filtered_elems.csv` with the required format:

`No.,Timestamp,Sequence Number, Attribute,Status,Data Type,Data Value.`

The numeric values of `RMS Current` and `RMS Voltage` are also sent to two Node-RED dashboard chart nodes. No custom timestamp is set for the chart: the dashboard uses the time at which the message reaches the chart node.

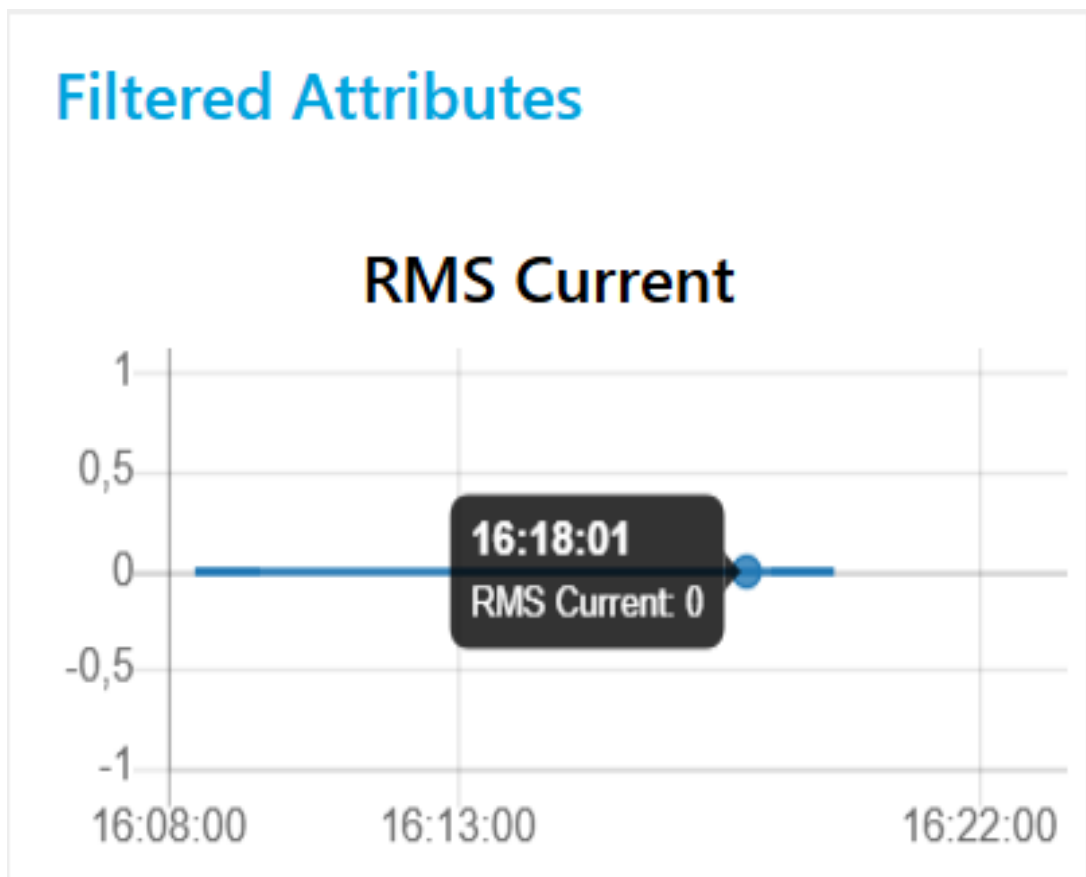


Figure 2: Node-RED dashboard chart for RMS Current.

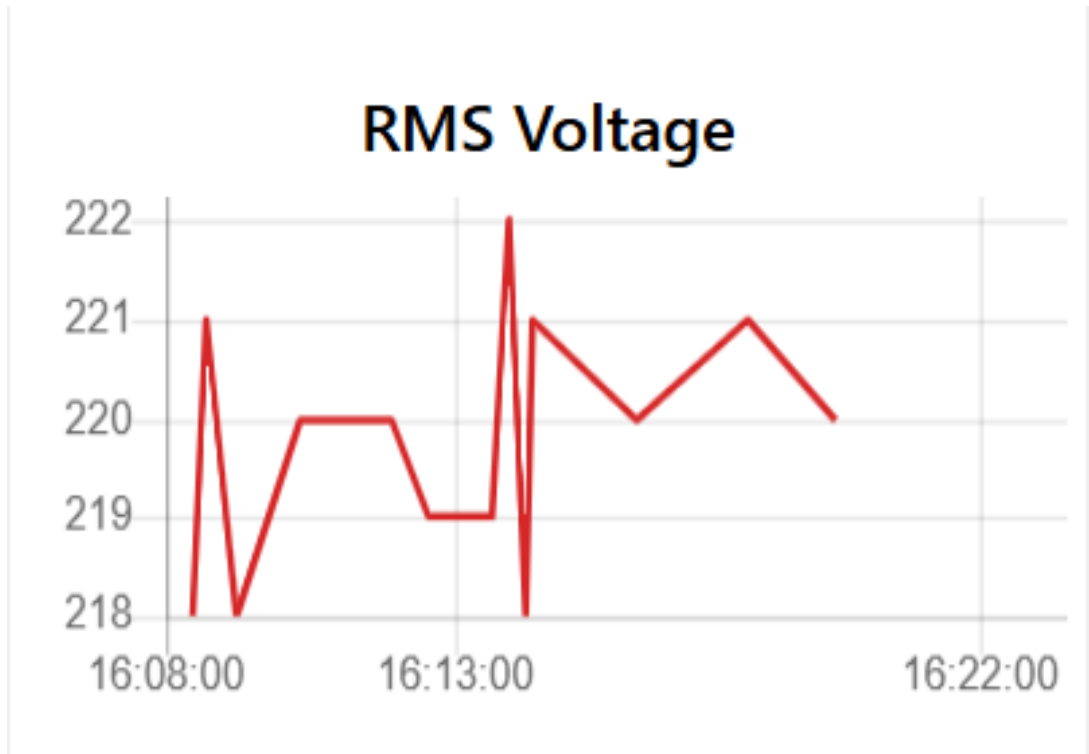


Figure 3: Node-RED dashboard chart for RMS Voltage.

For additional verification from the generated CSV files, the following plots were produced from the final outputs:

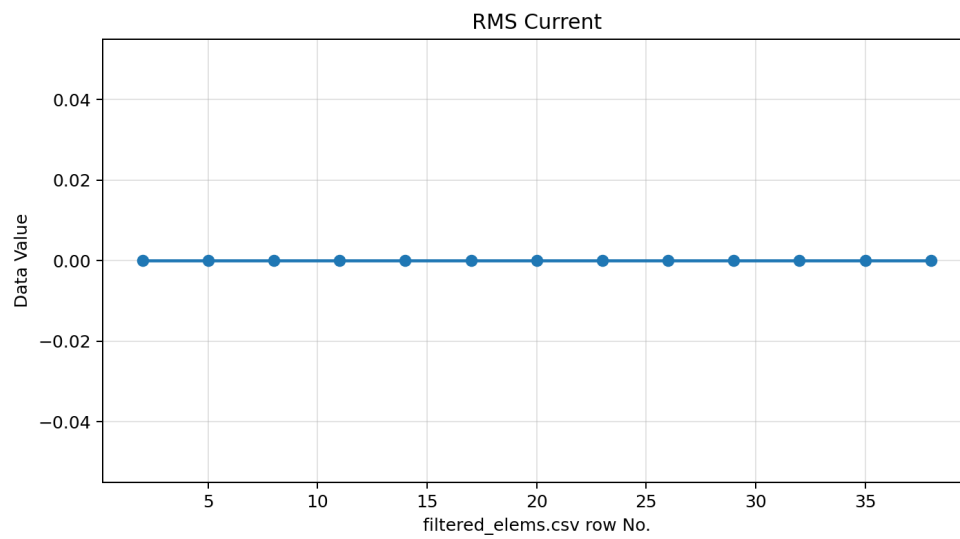


Figure 4: RMS Current values generated from `filtered_elems.csv`.

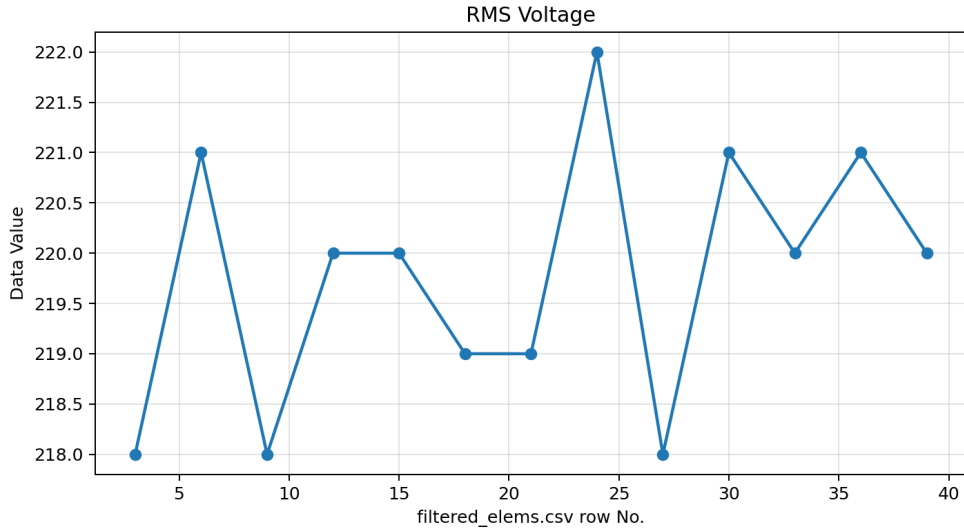


Figure 5: RMS Voltage values generated from `filtered_elems.csv`.

Link Status Branch

Packets are classified as Link Status packets by checking:

Packet Type = Link Status (0x08).

The function `parse Link Status` parses the `Command String` and extracts the list of links. For each link, it stores:

- **Source:** the row value of `Source Address ZigBee`;
- **Destination:** the link address;
- **Cost:** the link outgoing cost.

If a later Link Status packet updates an already known source/destination pair, the stored cost is updated. The output CSV preserves the order in which source/destination pairs were first received and processed. It is not sorted by source or destination.

The final CSV is written as:

No.,Source,Destination,Cost.

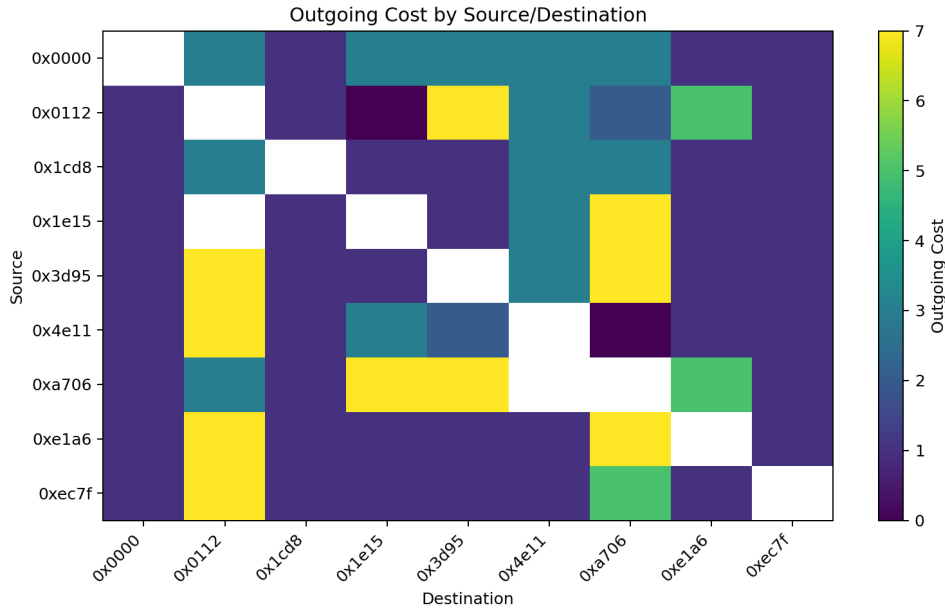


Figure 6: Outgoing cost matrix generated from `outgoing_cost.csv`.

ThingSpeak Branch

The ThingSpeak branch starts after the 200 received ID messages have been processed. The function `prepare ThingSpeak messages` reads the final Link Status state from the flow variables, then:

1. finds the smallest source address, interpreting addresses as hexadecimal numbers;
2. selects the destinations associated with that source;
3. sorts those destinations in ascending hexadecimal order;
4. sends each outgoing cost to ThingSpeak in `field1`.

The HTTP request URL is built as:

```
https://api.thingspeak.com/update?api_key=WRITE_API_KEY&field1=VALUE.
```

The API key is read from the environment variable `THINGSPEAK_WRITE_API_KEY`. It is not stored inside `flows.json`. The ThingSpeak 1 msg / 20s delay node is configured in queue mode and enforces one update every 20 seconds.

The public ThingSpeak channel used for the submission is:

```
https://thingspeak.mathworks.com/channels/3368448
```

The channel identifier is 3368448.

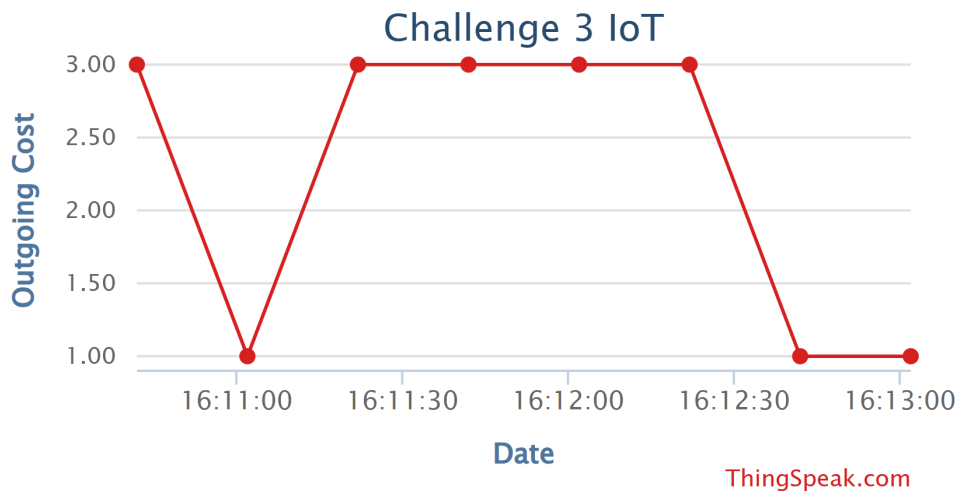


Figure 7: ThingSpeak chart for field1 outgoing cost updates.

Produced Deliverables

The Node-RED execution produces the following CSV files:

- `id_log.csv`, containing the generated ID messages;
- `filtered_elems.csv`, containing complete filtered attributes;
- `outgoing_cost.csv`, containing the final outgoing cost for each processed source/destination pair while preserving processing order.

The Node-RED flow is exported as JSON and delivered as `nodered.txt`.